BACSE101 Problem Solving using Python

PROJECT REPORT
on

University/Examination Portal – Admin/Client Access Management

Prepared by

K BARATH - 25BCE2402
KRISHNA VISHAL S – 25BCE2099
VIJAY R – 25BCE2314
S V ROHAN KAARTHICK - 25BCE2204
RAM GOVINDAN – 25BCE2456

Under the supervision of

Dr. PADMA PRIYA R

GitHub Repository URL:

https://github.com/Barath0303/University-Exam-Portal_PYTHON-PROJECT

School of Computer Science and Engineering
Vellore Institute of Technology, Vellore.

November 08, 2025

Table of Contents

Abstract

The project *"University/Examination Portal – Admin/Client Access Management"* is a Python-based system that manages student examination records through a command-line interface. It enables administrators to perform CRUD operations such as adding, updating, and viewing student data, while students can log in to check their results and grades.

The system uses MySQL for database management and a dictionary-based mode for offline access. Python libraries like mysql.connector, pandas, and faker are used for database connection, data handling, and generating sample records. This project highlights real-world application of data management, user interaction, and automation in educational portals.

1. Introduction

The *University/Examination Portal – Admin/Client Access Management* system is designed to simplify the management of student examination records using Python. It provides a structured command-line interface that allows the administrator to maintain and monitor student performance data efficiently.

The portal supports both **admin** and **client (student)** functionalities — where the admin can manage records, and students can view their individual results. The project integrates **MySQL** for secure data storage and also includes a **dictionary-based system** for offline usage.

This project emphasizes the use of Python programming concepts such as modular design, data structures, file handling, and database connectivity. It demonstrates practical application of CRUD operations, making it an efficient and user-friendly tool for academic data management.

1.1 Domain Information

This project falls under the **Education Management System** domain, specifically focusing on **Examination Data Management**. The system is designed to automate and organize student assessment records within a university environment.

The domain involves managing large sets of student-related data such as marks, grades, and performance statistics. By integrating database connectivity and automation using Python, the project addresses the need for efficient handling of academic records.

The use of MySQL ensures secure and scalable data storage, while the Python command-line interface allows easy interaction between the **admin** and **student** users. This domain encourages learning about real-world database applications, data analysis, and user access management.

1.2 Software Libraries Used

1. **mysql.connector**

- Used to connect Python with the **MySQL** database.
- Enables the execution of SQL queries for CRUD operations such as inserting, updating, deleting, and retrieving student records.

2. **pandas**

- Utilized for displaying and managing tabular data in a structured format.
- Simplifies the process of reading data from the database and presenting it in a clear, readable table for analysis and viewing.

3. **faker**

- Used to generate **synthetic student data** such as random names for testing and demonstration purposes.
- Helps populate the database or dictionary with realistic sample records without using real student information.

1.3 Contributions by Team Members

S. No.	Name of the Team Member	Register Number	Contribution / Module Implemented
1	**K BARATH**	25BCE2402	Implemented the MySQL database integration, connection setup, and CRUD operations (Add, Update, View, Delete). Handled CSV data loading, testing, and overall debugging of the project.
2	**KRISHNA VISHAL S**	25BCE2099	Developed the dictionary-based data management system for offline mode, including functions for viewing, adding, updating, and deleting student data. Integrated the module into the admin interface.
3	**VIJAY R**	25BCE2314	Designed and implemented the client (student) interface for result viewing and grade calculation. Created the logic for grade classification based on average marks.
4	**S V ROHAN KAARTHICK**	25BCE2204	Built the admin menu interface, linked all database and dictionary operations, and managed input handling, navigation, and user options.
5	**RAM GOVINDAN**	25BCE2456	Designed the main program structure and login system for both admin and student users. Managed user flow control, interface logic, and program termination functionality.

1.4 Challenges Faced

During the development of the **University/Examination Portal (Admin/Client Access Management)** project, the team encountered several challenges, including:

1. **Database Connectivity Issues:**
   Initial errors occurred while connecting MySQL with Python, mainly due to driver configuration and authentication settings. These were resolved by installing the correct MySQL connector and verifying connection parameters.

2. **Data Handling and Duplication:**
   Preventing duplicate entries when loading large datasets from CSV files required implementing validation checks before insertion.

3. **Data Integration Between SQL and Dictionary:**
   Ensuring both database and dictionary modes worked independently without conflicts demanded careful structuring of functions and menu options.

4. **Code Coordination Among Team Members:**
   Managing contributions from five members and merging all modules into one functional program posed coordination challenges. Proper version tracking and testing helped ensure consistency.

5. **Testing and Debugging:**
   Debugging user-input errors, menu logic, and query mismatches took time, especially when verifying CRUD operations across both modes.

2. Problem Statement and Objectives

In most educational institutions, examination management and result tracking are done manually or through isolated systems, which makes it difficult for administrators to manage student records efficiently and for students to access their results conveniently.
There is a need for a **centralized, user-friendly University/Examination Portal** that allows **administrators** to perform essential operations such as adding, updating, viewing, and deleting student data, while also enabling **students** to securely view their examination results and grades. This project addresses these challenges by developing a command-line-based management system that supports both **MySQL database integration** and an **offline dictionary-based mode** for flexible data handling.

The primary objectives of this project are:

1. To design and implement a **CLI-based University/Examination Portal** for both admin and client (student) access.

2. To enable **CRUD operations** (Create, Read, Update, Delete) for managing student data efficiently.

3. To integrate the system with a **MySQL database** for reliable data storage and retrieval.

4. To develop an **offline dictionary-based module** to simulate data management without external databases.

5. To generate and display **student performance reports**, including average marks and grade classification.

6. To promote **team collaboration**, dividing modules logically among team members for coordinated implementation.

3. Implementation

3.1 Features

The **University/Examination Portal (Admin/Client Access Management)** project includes the following main features:

1. Admin login and student login options.
2. Admin functionalities for student data management (CRUD operations).
3. Data storage and retrieval using **MySQL database**.
4. Offline data handling using a **Python dictionary** (no database required).
5. Student result and grade calculation.
6. A modular **menu-driven interface** for ease of use.

Each of these features is implemented as a separate function and can be accessed through the main or admin menu options.

3.2 Admin Login and Menu System

This module allows administrators to log in securely and perform various management tasks such as adding, viewing, updating, deleting student records, and accessing the dictionary-based module.

Code:

```
def admin_menu():
    while True:
        print("\n=== ADMIN MENU ===")
        print("1. Add Student")
        print("2. View Students")
        print("3. Update Student")
        print("4. Delete Student")
        print("5. Load Dataset")
        print("6. Logout")
        print("7. Use Dictionary Data (No SQL/CSV)")
        choice = input("Enter choice: ")
```

```
    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        update_student()
    elif choice == "4":
        delete_student()
    elif choice == "5":
        load_data()
    elif choice == "6":
        break
    elif choice == "7":
        dictionary_menu()
    else:
        print("Invalid choice.")
```

3.3 Add, View, Update, and Delete Student (CRUD Operations)

This feature enables the admin to perform **Create, Read, Update, and Delete** operations on student records stored in the MySQL database.

Code:

```
def add_student():
    conn = connect_db()
    cursor = conn.cursor()
    name = input("Enter name: ")
    gender = input("Enter gender: ")
    parental_education = input("Enter parental education: ")
    lunch = input("Enter lunch type: ")
    test_preparation = input("Enter test preparation course: ")
```

```
    math = int(input("Enter math score: "))
    reading = int(input("Enter reading score: "))
    writing = int(input("Enter writing score: "))

    cursor.execute("""
        INSERT INTO students
        (name, gender, parental_education, lunch, test_preparation_course, math_score, reading_score,
writing_score)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
    """, (name, gender, parental_education, lunch, test_preparation, math, reading, writing))

    conn.commit()
    conn.close()
    print("Student added successfully!")

def view_students():
    conn = connect_db()
    df = pd.read_sql("SELECT * FROM students", conn)
    df.set_index("id", inplace=True)
    print(df)
    conn.close()

def update_student():
    conn = connect_db()
    cursor = conn.cursor()
    student_id = int(input("Enter Student ID to update: "))

    print("\n1. Math Score\n2. Reading Score\n3. Writing Score\n4. All Scores")
    choice = input("Enter choice: ")
```

```
    if choice == "1":
        new_math = int(input("Enter new math score: "))
        cursor.execute("UPDATE students SET math_score=%s WHERE id=%s", (new_math, student_id))
    elif choice == "2":
        new_reading = int(input("Enter new reading score: "))
        cursor.execute("UPDATE students SET reading_score=%s WHERE id=%s", (new_reading, student_id))
    elif choice == "3":
        new_writing = int(input("Enter new writing score: "))
        cursor.execute("UPDATE students SET writing_score=%s WHERE id=%s", (new_writing, student_id))
    elif choice == "4":
        new_math = int(input("Enter new math score: "))
        new_reading = int(input("Enter new reading score: "))
        new_writing = int(input("Enter new writing score: "))
        cursor.execute("""
            UPDATE students
            SET math_score=%s, reading_score=%s, writing_score=%s
            WHERE id=%s
        """, (new_math, new_reading, new_writing, student_id))
    else:
        print("Invalid choice.")
        conn.close()
        return

    conn.commit()
    conn.close()
    print("Student record updated successfully!")
```

```
def delete_student():
    conn = connect_db()
    cursor = conn.cursor()
    student_id = int(input("Enter Student ID to delete: "))
    cursor.execute("DELETE FROM students WHERE id=%s", (student_id,))
    conn.commit()
    conn.close()
    print("Student deleted successfully!")
```

3.4 Student Login and Result Viewing

Students can log in and view their examination results, including their marks and calculated grade based on average scores.

Code:

```
def view_result():
    conn = connect_db()
    student_id = int(input("Enter your Student ID: "))
    df = pd.read_sql(f'SELECT * FROM students WHERE id={student_id}", conn)
    if df.empty:
        print("Student not found.")
    else:
        print("\n=== STUDENT RESULT ===")
        print(df)
        avg = (df['math_score'].iloc[0] + df['reading_score'].iloc[0] + df['writing_score'].iloc[0]) / 3
        print(f"\nAverage Score: {avg:.2f}")
        if avg >= 90:
            print("Grade: S")
        elif avg >= 80:
            print("Grade: A")
```

```
    elif avg >= 70:
        print("Grade: B")
    elif avg >= 60:
        print("Grade: C")
    else:
        print("Grade: D")
conn.close()
```

3.5 Dictionary-Based Data Management (Offline Mode)

This module performs all CRUD operations using a **Python dictionary** instead of a database or CSV file, useful when MySQL is unavailable.

Code:

```
students_dict = {
  1: {"name": "Aarav Sharma", "gender": "Male", "parental_education": "Bachelor's", "lunch": "standard", "test_preparation_course": "none", "math_score": 85, "reading_score": 78, "writing_score": 80},
  2: {"name": "Diya Patel", "gender": "Female", "parental_education": "Master's", "lunch": "free/reduced", "test_preparation_course": "completed", "math_score": 92, "reading_score": 89, "writing_score": 95},
  ...
}

def show_dict_data():
  print("\n=== DICTIONARY STUDENT DATA ===")
  for sid, details in students_dict.items():
    print(f"ID: {sid} | Name: {details['name']} | Math: {details['math_score']} | Reading: {details['reading_score']} | Writing: {details['writing_score']}")

def add_dict_student():
  new_id = len(students_dict) + 1
  name = input("Enter name: ")
  gender = input("Enter gender: ")
```

```
    parental_education = input("Enter parental education: ")
    lunch = input("Enter lunch type: ")
    test_preparation = input("Enter test preparation course: ")
    math = int(input("Enter math score: "))
    reading = int(input("Enter reading score: "))
    writing = int(input("Enter writing score: "))


    students_dict[new_id] = {
        "name": name,
        "gender": gender,
        "parental_education": parental_education,
        "lunch": lunch,
        "test_preparation_course": test_preparation,
        "math_score": math,
        "reading_score": reading,
        "writing_score": writing
    }
    print("Student added successfully to dictionary!")
```

3.6 Dataset Loading (CSV → MySQL)

This feature loads and inserts student data from the **Kaggle dataset** into the MySQL database, generating fake names using the Faker library.

Code:

```
def load_data():
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM students")
    existing_count = cursor.fetchone()[0]
```

```
if existing_count > 0:
    print(f"Database already has {existing_count} records. Skipping data load.")
    conn.close()
    return

df = pd.read_csv("StudentsPerformance.csv")
df['name'] = [fake.name() for _ in range(len(df))]

for _, row in df.iterrows():
    cursor.execute("""
        INSERT INTO students
        (name, gender, parental_education, lunch, test_preparation_course, math_score, reading_score, writing_score)
        VALUES (%s,%s,%s,%s,%s,%s,%s,%s)
    """, (row['name'], row['gender'], row['parental level of education'], row['lunch'], row['test preparation course'], row['math score'], row['reading score'], row['writing score']))

conn.commit()
conn.close()
print("Data loaded successfully into MySQL")
```

4.Demo Screenshots

```
=== UNIVERSITY EXAM PORTAL ===
1. Admin Login
2. Student Login
3. Exit
Enter your choice:
```

```
=== ADMIN MENU ===
1. Add Student
2. View Students
3. Update Student
4. Delete Student
5. Load Dataset
6. Logout
7. Use Dictionary Data (No SQL/CSV)
Enter choice: 1
Enter name: Visha;
Enter gender: Male
Enter parental education: bachelor's
Enter lunch type: standard
Enter test preparation course: none
Enter math score: 99
Enter reading score: 88
Enter writing score: 89
Student added successfully!
```

id	name	gender	parental_education	lunch	test_preparation_course	math_score	reading_score	writing_score
1	Diane Anderson	female	bachelor's degree	standard	none	72	72	74
2	Brian Hayes	female	some college	standard	completed	69	90	88
3	Norma Mason	female	master's degree	standard	none	90	95	93
4	Brandon Davila	male	associate's degree	free/reduced	none	47	57	44
5	Melissa Weber	male	some college	standard	none	76	78	75
...	...	...	...	...	...	...	...	...
998	Brandon Miller	female	high school	free/reduced	completed	59	71	65
999	Timothy Miller	female	some college	standard	completed	68	78	77
1000	Brian Vang DDS	female	some college	free/reduced	none	77	86	86
1001	Tamizh	Female	bachelor's degree	standard	completed	100	100	99
1003	Visha;	Male	bachelor's	standard	none	99	88	89

[1002 rows x 8 columns]

```
Enter choice: 3
Enter Student ID to update: 1003

What do you want to update?
1. Math Score
2. Reading Score
3. Writing Score
4. All Scores
Enter choice: 1
Enter new math score: 89
Student record updated successfully!
```

```
Enter choice: 4
Enter Student ID to delete: 1003
Student deleted successfully!
```

```
=== DICTIONARY DATA MENU ===
1. View All Students
2. Add Student
3. Update Student
4. Delete Student
5. Back to Admin Menu
Enter choice: 1
           name  gender parental_education        lunch test_preparation_course  math_score  reading_score  writing_score
id
1   Aarav Sharma    Male         Bachelor's     standard                 none          85             78             80
2     Diya Patel  Female           Master's  free/reduced            completed          92             89             95
3    Rohan Singh    Male         Highschool     standard                 none          68             74             70
4   Ananya Gupta  Female         Bachelor's     standard            completed          88             92             90
5    Krish Mehta    Male       Some college     standard                 none          60             65             70
6   Ishita Reddy  Female         Highschool  free/reduced                 none          75             72             74
7    Vikram Nair    Male           Master's     standard            completed          95             98             94
8     Meera Iyer  Female       Some college     standard                 none          78             82             80
9      Arjun Rao    Male         Bachelor's     standard            completed          84             88             86
10      Neha Das  Female           Master's  free/reduced                 none          70             75             72
```

```
=== CLIENT MENU ===
1. View My Result
2. Logout
Enter choice: 1
Enter your Student ID: 1001
C:\Users\barat\OneDrive\Desktop\Python Project\university_portal.py:135: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or
 sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df = pd.read_sql(f"SELECT * FROM students WHERE id={student_id}", conn)

=== STUDENT RESULT ===
     id    name  gender parental_education     lunch test_preparation_course  math_score  reading_score  writing_score
0  1001  Tamizh  Female  bachelor's degree  standard              completed         100            100             99

Average Score: 99.67
Grade: S
```

5. Conclusion

The **University Examination Portal** successfully demonstrates an efficient and interactive system for managing student examination records. It provides functionalities for both **administrators** (to add, update, delete, and view student data) and **students** (to view their scores and grades).
The use of **Python**, **MySQL**, and **Faker** ensures a balance between real-time database handling and simulated test data generation. Additionally, the **dictionary-based approach** enhances flexibility by allowing offline operations without requiring external storage or datasets.
Overall, the system meets its objectives by offering a simple, scalable, and modular solution for academic record management.